

Primer paso: Propuesta de TFM

Para comenzar el proceso, debéis elaborar y entregar una propuesta de proyecto a través del recurso habilitado en el Campus.

Se trata de un documento breve (máximo 2–3 páginas, siendo una página suficiente) que debe incluir:

- Descripción de la propuesta.
- Motivación del proyecto.
- Objetivos finales que se persiguen.

Plazo y recomendaciones:

El plazo para presentar la propuesta finaliza el **24/03/2026**.

Aunque el sistema permitirá entregas posteriores para quienes necesiten más tiempo, es fundamental no dejarlo para el último momento. El TFM requiere planificación, y cuanto antes obtengáis la validación de vuestra idea, mejores serán los resultados del trabajo final.

Propuesta Ansible

Arquitectura de **autorreparación inteligente (Self-healing infrastructure)**.

La clave del proyecto es que los sistemas tradicionales de respuesta (IPS) son rígidos. Si el ataque no encaja exactamente en el script, el script falla. La IAG (Generative AI) aporta el "razonamiento" para interpretar la alerta de Snort y decidir qué Playbook de Ansible es el más adecuado o, incluso, cómo parametrizarlo dinámicamente.

1. El Flujo de Trabajo (The Pipeline)

El sistema no debe ser un bucle infinito de "espera", sino una arquitectura dirigida por eventos (*Event-Driven*).

- Detección:** Snort detecta un patrón sospechoso (ej. un *SQL Injection* o un *Syn Flood*) y genera una alerta en su log (`/var/log/snort/alert`).
- Ingesta e Inteligencia:** Un script intermedio (Orquestador en Python) detecta la nueva línea en el log y envía el contexto (el log de Snort + el estado actual del sistema) a un LLM (puede ser local como Llama 3 vía Ollama o una API externa).
- Razonamiento y Selección:** El LLM analiza la gravedad y el tipo de ataque. Consulta una base de datos de "Capacidades de Ansible" (RAG) y decide qué acción tomar.
- Ejecución:** El Orquestador lanza **Ansible Runner** pasando los parámetros generados por la IA (ej. la IP atacante a bloquear).
- Feedback:** Se verifica si el ataque persiste y se guarda la experiencia en una base de datos para mejorar futuras respuestas.

2. Arquitectura Técnica

Para un TFM, la arquitectura debe ser robusta. Te sugiero una estructura de **cuatro capas**:

A. Capa de Monitorización (Snort)

- **Configuración:** Snort en modo NIDS.
- **Salida:** Configurar Snort para que escupa logs en formato **JSON**. Esto es vital porque los LLMs procesan mucho mejor el JSON estructurado que el texto plano.

B. Capa de Razonamiento (GenAI + RAG)

La idea es no preguntar a la IA "qué hago" a secas, sino usar **RAG (Retrieval-Augmented Generation)**:

- **Vector DB:** Guardar los Playbooks de Ansible y documentación de seguridad (OWASP, etc.) en una base de datos vectorial (como ChromaDB o Pinecone).
- **Prompt Engineering:** El prompt dirá: *"Dado este log de Snort [LOG] y sabiendo que tienes estos Playbooks disponibles [RAG_CONTEXT], genera el comando JSON para ejecutar el Playbook que mitigue el ataque"*.

C. Capa de Actuación (Ansible)

- **Ansible Automation Platform o Ansible Core:** Se usará **ansible-runner** (la librería de Python) para ejecutar playbooks de forma programática.
- **Playbooks Modulares:** Crear playbooks para:
 - **quarantine_ip.yml:** Bloquear una IP en **iptables** o **nftables**.
 - **rate_limit.yml:** Modificar la configuración de Nginx para limitar peticiones.
 - **service_restart.yml:** Reiniciar servicios afectados.

D. Capa de Observabilidad

- Un dashboard (Streamlit o Flask) donde se vea en tiempo real: Ataque detectado -> Razonamiento de la IA -> Acción de Ansible ejecutada.

3. Objetivo final

Al terminar el TFM, se habrá construido un **SOC (Security Operations Center) Autónomo de Nivel 1**. El resultado tangible será:

1. **Reducción del MTTR (Mean Time To Remediation):** De minutos (humano) a milisegundos (IA+Ansible).
2. **Mitigación Adaptativa:** A diferencia de un script fijo, si Snort detecta un ataque nuevo, la IA puede intentar una combinación de reglas de firewall y cambios en la configuración del servidor que no estaban previstos explícitamente.
3. **Explicabilidad:** El sistema no solo bloquea, sino que genera un informe de **por qué** tomó esa decisión basándose en el log de Snort, algo fundamental para auditorías de seguridad.

Propuesta de Trabajo Fin de Máster (TFM)

Título: Implementación de un Asistente de Viajes Inteligente mediante Model Context Protocol (MCP) y RAG con Interfaz Omnicanal (WhatsApp/API).

1. Descripción de la propuesta

El proyecto consiste en el diseño y desarrollo de un ecosistema de asistencia al viajero basado en Inteligencia Artificial Generativa. A diferencia de los chatbots convencionales, este sistema utilizará el **Model Context Protocol (MCP)** como eje central para la interoperabilidad de herramientas.

El asistente permitirá a los usuarios planificar, gestionar y monitorizar sus viajes en tiempo real. La arquitectura se basará en:

- **Orquestación de Herramientas (MCP):** Uso de servidores MCP para conectar el modelo de lenguaje con fuentes de datos externas (APIs de vuelos, buscadores de alojamiento y bases de datos locales).
- **RAG Dinámico:** Implementación de una base de conocimientos para consultas sobre requisitos legales, visados y guías culturales actualizadas.
- **Interfaz de Usuario:** Despliegue principal mediante la API de WhatsApp para garantizar accesibilidad, respaldado por una API REST para posibles integraciones web.

2. Motivación del proyecto

La planificación de viajes suele ser un proceso fragmentado que obliga al usuario a saltar entre múltiples aplicaciones (reservas, mapas, documentos, gestión de gastos). La IA generativa ofrece la capacidad de unificar estas experiencias, pero suele fallar en la precisión de datos en tiempo real y en la ejecución de acciones concretas.

La aparición de **MCP** por parte de Anthropic (y su adopción por la comunidad) resuelve el problema de la "rigidez" de las herramientas tradicionales. La motivación técnica reside en explorar cómo este protocolo facilita la escalabilidad de funciones (tools) sin necesidad de reescribir la lógica del agente cada vez que se añade un servicio. Además, el uso de **WhatsApp** como interfaz responde a la necesidad de ofrecer una herramienta útil en condiciones de movilidad, donde la sencillez y la baja latencia son críticas.

3. Objetivos finales

El objetivo general es construir un sistema de agentes capaz de razonar sobre las necesidades del viajero y ejecutar acciones precisas mediante herramientas conectadas.

Objetivos específicos:

1. **Arquitectura de Herramientas (Tools):** Desarrollar servidores MCP que permitan al LLM realizar búsquedas de vuelos y alojamientos mediante APIs externas.
2. **Módulo de Conocimiento (RAG):** Implementar una base de datos vectorial para proveer información precisa sobre documentación necesaria por país y recomendaciones turísticas personalizadas.
3. **Gestión de Finanzas y Datos:** Integrar una herramienta de base de datos (ej. PostgreSQL o SQLite vía MCP) para el tracking de gastos del usuario y la gestión de recordatorios de viaje.
4. **Integración Omnicanal:** Establecer un flujo de comunicación bidireccional entre el motor de IA y WhatsApp, gestionando el historial de conversación y el contexto del usuario.
5. **Evaluación de Eficacia:** Medir la tasa de éxito del agente en la resolución de tareas complejas (ej. "Encuentra un vuelo a Japón, dime si necesito visa y guarda un presupuesto de 1500€").

Sugerencias adicionales para tu TFM:

- **Tecnologías sugeridas:** LangGraph o CrewAI para la orquestación de agentes, Pinecone o Weaviate para la base de datos vectorial, y Twilio o la API oficial de Meta para WhatsApp.
 - **Factor Diferencial:** Podrías incluir un "modo offline" donde el RAG proporcione información básica guardada previamente si el usuario tiene poca conexión (común en viajes).
-

Propuesta de Trabajo Fin de Máster (TFM)

Título: Sistema Agéntico de Asistencia al Viajero: Integración de MCP, RAG y Herramientas de Persistencia en Interfaces de Mensajería Instantánea.

1. Descripción de la propuesta

El proyecto consiste en el desarrollo de un asistente inteligente basado en IA Generativa diseñado para acompañar al usuario en todas las etapas de un viaje. La solución se despliega a través de canales de **mensajería instantánea** (WhatsApp, Telegram, etc.), actuando como una interfaz conversacional única que centraliza diversos servicios.

La arquitectura técnica se apoya en tres pilares:

1. **Model Context Protocol (MCP):** Para la conexión estandarizada con herramientas de búsqueda y bases de datos.
2. **RAG (Retrieval-Augmented Generation):** Para el acceso a información normativa y turística veraz.
3. **Orquestación de Agentes:** Para la gestión de flujos de trabajo complejos y persistencia de datos.

2. Motivación del proyecto

La planificación de viajes padece de una alta fragmentación. El usuario debe consultar múltiples fuentes para documentación, precios y control de gastos. El uso de **MCP** representa una innovación radical en la ingeniería de prompts y el uso de herramientas, ya que permite que el LLM interactúe con el mundo real de forma modular.

Este proyecto busca demostrar que es posible construir un "operador de viajes" personal que resida en el bolsillo del usuario, minimizando la fricción de entrada mediante el uso de interfaces de chat ya familiares, y garantizando la precisión de los datos mediante técnicas avanzadas de recuperación y ejecución de código.

3. Objetivos finales (MVP de 4 funcionalidades)

El objetivo general es validar la viabilidad de un asistente agéntico mediante la implementación de las siguientes capacidades core; cada una de las funcionalidades estará enfocada en una primera instancia a vuelos y para ser testeada en uno o dos países aún por definir:

- **Módulo de Consultoría Normativa (RAG):** Implementar un sistema de recuperación que permita al usuario consultar requisitos legales, visados y alertas sanitarias de cualquier destino. Se utilizará una base de datos vectorial para

asegurar que las respuestas se basen en documentos oficiales y no en alucinaciones del modelo.

- **Buscador Logístico en Tiempo Real (MCP + Tools):** Desarrollar la capacidad del asistente para conectarse a APIs externas de vuelos y alojamiento. Mediante el protocolo MCP, el asistente podrá realizar búsquedas activas y comparar opciones vigentes según los criterios del usuario.
- **Gestor de Finanzas del Viajero (Persistencia):** Crear una herramienta de registro de gastos en lenguaje natural. El sistema deberá ser capaz de procesar entradas como "*Anota 20€ en transporte*", almacenarlas en una base de datos estructurada y generar reportes de presupuesto bajo demanda.
- **Agente de Itinerario y Recordatorios:** Implementar una lógica de gestión temporal que permita al usuario delegar recordatorios críticos (vuelos, check-ins, trenes). El sistema enviará notificaciones proactivas a la plataforma de mensajería para asistir al usuario en tiempo real durante su trayecto.